Title: Design and Analysis of Algorithms (MnC) SEM III core
Course code: MC213
Structure: 3-1-0-4
Prerequisites: Programming, Data Structures
Course contents:

1. Notions of problems and algorithms, analysis and asymptotic notation.
2. Paradigms of iterative, divide & conquer, greedy algorithms and dynamic programming (covered via problems and algorithms).
3. Sorting algorithms (insertion, selection, bubble, merge, quick, heap) and analysis. counting sort, radix sort.
4. Searching: linear search, binary search, ranks in linear lists.
5. Greedy algorithms: fractional knapsack problem, Huffman codes,
6. Dynamic programming: search-frequency weighted BST, all-pairs shortest paths, matrix chain multiplication, longest common subsequence.
7. Graph algorithms: BFS, DFS, Dijkstras algorithm for single source shortest paths, minimum spanning trees (Prims, Kruskals, and other algorithms), network flows.
8. String matching algorithms: Rabin-Karp algorithm, KMP algorithm.
9. NP-completeness and basics of complexity theory: concepts of reductions, and NP-completeness.
10. Some standard methods of handling NPC problems: approximation algorithms, special cases, heuristics.

Suggested textbook/references:

1. Introduction to Algorithms - Cormen, Leiserson, Rivest and Stein.
2. Algorithm Design - Kleinberg and Tardos
3. Algorithms - Robert Sedgewick and Kevin Wayne
4. The Design and Analysis of Computer Algorithms - Aho, Hopcroft and Ullman

Grading policy: Every student must participate in two in-semester examinations and one final examination. Their grade at the conclusion of the course will be determined by their performance in these assessments. Additionally, there will be 4-5 assignments (either programming or theoretical) designed to evaluate their programming and algorithm design abilities. The tentative weight distribution, subject to slight variation, is as follows: In-sem I (30%), In-sem II (30%), and End-sem (40%). Attendance follows institute policy.
Course outcome:

- Algorithmic Knowledge: Understand fundamental algorithms, including sorting, searching, and graph algorithms.
- Algorithm Design Skills: Apply techniques like divide and conquer, dynamic programming, and greedy algorithms to solve computational problems.
- Complexity Analysis: Analyze algorithms using Big-O notation to determine time and space complexity.
- Problem-Solving Proficiency: Develop skills in solving algorithmic problems through logical and structured thinking.
- Implementation Ability: Implement algorithms in programming assignments to translate theoretical concepts into practical solutions.
- Critical Thinking: Evaluate and compare different algorithms based on their theoretical analysis and practical performance.

P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12
✖	✖	✖									✖